
Thinking Globally, Acting Locally: Graph Neural Networks for Decentralised MEC Offloading

Anirudh M

Department of Computer Science
BITS Pilani, Hyderabad Campus

f20240020@hyderabad.bits-pilani.ac.in

Harshul Agarwal

Department of Computer Science
BITS Pilani, Hyderabad Campus

f20240177@hyderabad.bits-pilani.ac.in

Abstract

Multi-agent computation offloading in Non-Terrestrial Network (NTN) empowered Multi-access Edge Computing (MEC) systems requires agents to coordinate shared UAV resources under partial observability. Existing decentralised approaches struggle with this coordination, while centralised methods impose heavy infrastructure overhead. To bridge this gap, we propose GNN-IL: a bipartite weighted GraphSAGE encoder combined with a shared Dueling DQN, trained in a fully decentralised manner. GNN-IL achieves up to a 41.1% cost reduction over standard Independent Learning and outperforms Centralised Training Decentralised Execution (CTDE) without requiring a central controller at any stage. Controlled ablation experiments reveal that these gains are driven by gradient coupling via parameter sharing rather than richer observations alone. Furthermore, we establish that the fundamental bottleneck in independent learning is spatial rather than temporal, and identify a bimodal convergence phase transition that resolves as agent count scales.

1 Introduction

Multi-agent computation offloading in Non-Terrestrial Network (NTN) empowered Multi-access Edge Computing (MEC) systems presents a fundamental coordination challenge: multiple User Equipments (UEs) compete for shared UAV edge server resources, yet each agent observes only its own local state.

Fatima et al. [2024] addressed this with two Multi-Agent Distributed Deep Reinforcement Learning (MADDRL) frameworks: Centralised Training Decentralised Execution (CTDE) and Independent Learning (IL), demonstrating that CTDE outperforms IL by 11% through shared experience pooling, at the cost of centralised training infrastructure.

This raises a natural question: **can a fully decentralised method match or exceed CTDE by giving agents better spatial awareness, rather than by centralising their training?** We propose **GNN-IL**: a bipartite weighted GraphSAGE encoder combined with a shared Dueling DQN, trained in a fully decentralised manner. At $M = 30$ UEs, GNN-IL reduces average cost by **41.1%** over IL and outperforms CTDE across all evaluated agent counts, without a central controller at any stage.

Beyond the performance result, this work makes four contributions. First, we show that graph structure and parameter sharing are both necessary. GNN-IL-NoShare, which provides GNN-enriched

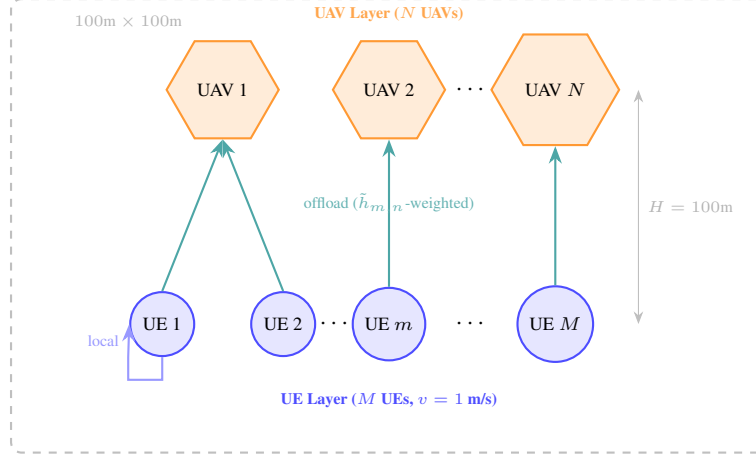


Figure 1: NTN-MEC system model. M mobile UEs (blue) compete for N UAV edge servers (amber) over channel-gain weighted links. Each UE either offloads its task to a UAV or processes it locally per slot.

observations without shared weights, performs worse than vanilla IL. Thus, gradient coupling is a vital part of the good performance.

Second, we establish that the bottleneck in independent learning is spatial, not temporal - frame stacking hurts IL and a GRU encoder diverges, ruling out temporal memory as an explanation.

Third, a controlled fairness analysis reveals that information utility depends on architectural mechanism: the same congestion feature $B_n(i)$ is transformative for CTDE, irrelevant for IL, and naturally integrated by GNN-IL.

Fourth, GNN-IL exhibits a bimodal convergence phase transition at intermediate scale ($M = 20$) that resolves as agent count grows, with convergence becoming more reliable precisely where the coordination problem is hardest.

The remainder of this paper is structured as follows. Section 2 describes the system model and Dec-POMDP formulation. Section 3 details all methods. Section 4 describes the experimental setup. Section 5 presents results and ablations. Section 6 discusses deployment tradeoffs and limitations. Section 7 concludes with future work directions.

2 System Model and Problem Formulation

2.1 Environment and Task Model

We consider an NTN-empowered MEC system deployed over a $100\text{m} \times 100\text{m}$ area, comprising M User Equipments (UEs) and N UAVs equipped with edge servers, as illustrated in Fig. 1. Following the model in Fatima et al. [2024], the system operates in discrete time slots of $\Delta = 0.2\text{s}$. UEs move at 1 m/s in random directions. At each slot i , each UE generates a task with probability $P = 0.3$. Tasks are defined by size ($2 - 5\text{ Mbits}$), processing density ($0.297\text{ gigacycles/Mbit}$), and a 10-slot deadline.

To process a task, UE m selects an action $a_m(i) \in \mathcal{C} = \{0, 1, \dots, N\}$, denoting either local processing (0) or offloading to a specific UAV.

Conscious simplifications. The base paper additionally includes a LEO satellite, $N = 4$ UAVs, and 1000 training episodes. We omit the LEO satellite to isolate UAV-to-UAV coordination dynamics and reduce the action space. We fix $N = 2$ and train for 500 episodes; at $M = 10$, this still yields 150,000

task experiences, proportionally matching the base paper’s scale. We vary $M \in \{5, 10, 20, 30, 40\}$ for explicit scalability analysis.

2.2 Communication, Computation, and Cost

We adopt the distance-dependent path loss and queue dynamics of Fatima et al. [2024]. Uplink transmission rates are governed by a standard Shannon capacity model (1 MHz bandwidth, 0.1 W transmit power, -102 dBm noise). UEs and UAVs maintain respective processing and transmission queues. A critical derived quantity is $B_n(i)$: the number of active queues at UAV n at slot i , which dictates how the UAV’s 2.5 GHz CPU capacity is shared equally among offloaded tasks (analysed further in Section 5.6).

The system cost for UE m at slot i , denoted $\Psi_m(i)$, is the equally weighted sum of its total processing/transmission delay $t_m(i)$ and energy consumption $e_m(i)$:

$$\Psi_m(i) = 0.5 \cdot t_m(i) + 0.5 \cdot e_m(i). \quad (1)$$

The global objective is to minimise the long-run average cost per UE.

2.3 Dec-POMDP Formulation

The high-dimensional, multi-agent nature of the system makes it a natural fit for a Decentralised Partially Observable Markov Decision Process (Dec-POMDP). Each UE m acts as an independent agent making offloading decisions based solely on a local observation vector $o_m(i)$ of dimension $5 + 4N$, constructed as follows:

Table 1: Observation vector components for each UE m at slot i

Feature	Dim	Description
$\mathbf{p}_m(i)$	2	Normalised UE position (x, y)
$\mathbf{q}_1, \dots, \mathbf{q}_N$	$2N$	Normalised UAV positions
$D_m(i)$	1	Task size (0 if no task)
$\delta_m^{\text{comp}}(i)$	1	Local computation queue wait
$\delta_m^{\text{tran}}(i)$	1	Transmission queue wait
$l_{n,m}^{\text{edge}}$	N	Per-UAV edge queue residual
$B_n(i)/M^\dagger$	N	Normalised active queues at UAV n

[†]Initially absent; added after the fairness analysis in Section 5.6.

At $N = 2$, the observation dimension is 13. The agent receives a reward $r_m(i) = -\Psi_m(i)$ if the task is completed before the deadline, and a large constant penalty $r_m(i) = -\Psi_{\text{drop}}$ if the task is dropped. The discount factor for future rewards is $\gamma = 0.9$.

3 Methods

3.1 Shared Architecture: Dueling DQN

All methods in this work are built on a common Dueling Deep Q-Network (DQN) [Wang et al., 2016] backbone. The network splits its final layer into two streams: a value stream estimating the state value $V(s)$, and an advantage stream estimating the relative advantage $A(s, a)$ of each action. Q-values are recovered as:

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s, a'), \quad (2)$$

improving stability by decoupling state evaluation from action selection.

Each stream consists of a fully connected layer followed by ReLU activation and a linear output layer, with hidden dimension 128.

A separate but identical target network is maintained and updated via hard copy every 20 steps to stabilise training. All methods use Adam with learning rate 10^{-3} , discount factor $\gamma = 0.9$, and minibatch size 32. Exploration follows an ε -greedy policy.

3.2 Independent Learning (IL)

Independent Learning serves as our primary baseline, implemented fully faithfully to Algorithm 2 of Fatima et al. [2024].

Because the base paper’s IL algorithm inherently lacks the Gated Recurrent Unit (GRU) module used in their CTDE framework, our implementation likewise omits recurrent processing. As a supplementary experiment, we attempted to augment IL with a GRU but observed monotonically increasing training loss ($1.29 \rightarrow 3.42$ over 500 episodes) due to exploding gradients over time.

This confirms that the IL baseline does not benefit from temporal memory, a finding we return to in Section 5.3.

3.3 Centralised Training Decentralised Execution (CTDE)

Our CTDE implementation follows Algorithm 1 of Fatima et al. [2024] faithfully, with a single structural modification: the GRU load prediction module is explicitly omitted.

This omission is a deliberate symmetric choice. By ensuring that neither IL nor our CTDE baseline utilises recurrent processing, any observed performance differences between the two can be strictly attributed to the centralised experience pooling. This is analyzed further in Section 5.6.

3.4 GNN-Augmented Independent Learning (GNN-IL)

GNN-IL is the proposed method. It augments the IL framework with a bipartite Graph Neural Network encoder that gives each agent a spatially-enriched embedding, and shares all parameters - encoder and DQN - across all M agents.

3.4.1 Graph Construction

At each time slot i , the system state is represented as a bipartite graph $\mathcal{G}(i) = (\mathcal{V}_{\text{UE}} \cup \mathcal{V}_{\text{UAV}}, \mathcal{E})$.

UE nodes $\mathcal{V}_{\text{UE}}$ carry the raw observation vector $o_m(i)$ as node features.

UAV nodes $\mathcal{V}_{\text{UAV}}$ carry a three-dimensional feature vector: normalised position $(X_n/L, Y_n/W)$ and normalised active queue count $B_n(i)/M$ that encodes the current congestion state.

Each UE m is connected to every UAV n by a directed edge with weight equal to the normalised channel gain $\tilde{h}_{m,n}(i)$, reflecting link quality at the current positions of both nodes.

3.4.2 Two-Layer Weighted GraphSAGE

The GNN encoder is a two-layer weighted GraphSAGE [Hamilton et al., 2017]. In the first layer, each UAV node aggregates information from all connected UEs, weighted by channel gain:

$$\mathbf{h}_n^{\text{UAV}} = \text{MLP}\left(\left[\mathbf{f}_n^{\text{UAV}} \parallel \sum_{m \in \mathcal{M}} \tilde{h}_{m,n}(i) \cdot o_m(i)\right]\right), \quad (3)$$

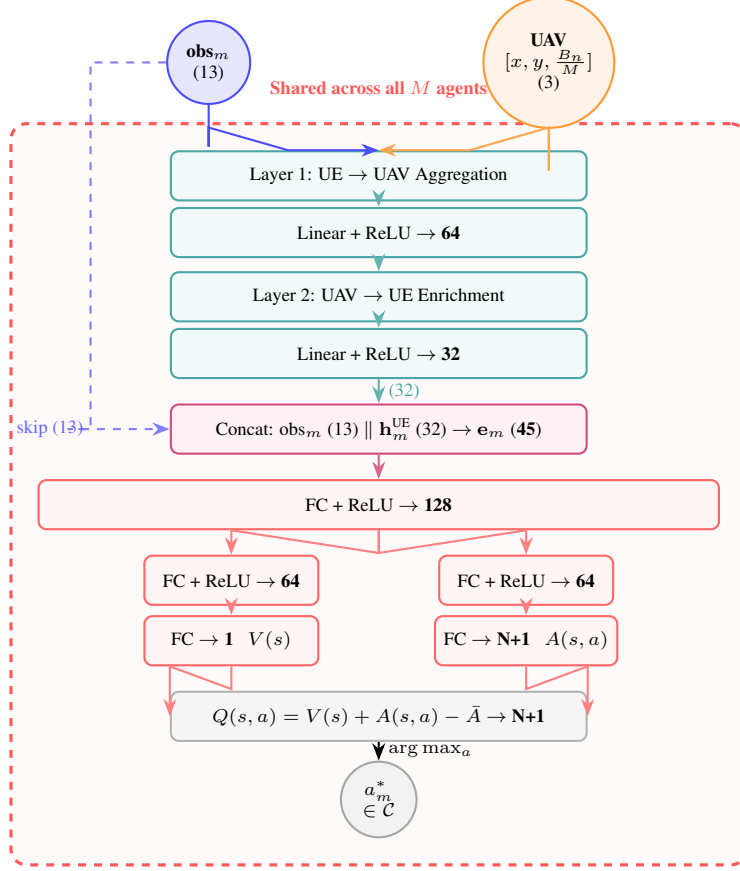


Figure 2: GNN-IL architecture. Input circles show raw features entering the graph encoder. Two GraphSAGE aggregation layers produce a 32-dimensional UE embedding, concatenated with the raw observation via a skip connection to form $\mathbf{e}_m \in \mathbb{R}^{45}$. The Dueling DQN maps this to Q-values via separate value and advantage streams. All components inside the dashed box are shared across all M agents.

producing a $d_{\text{hidden}} = 64$ dimensional embedding per UAV. In the second layer, each UE node aggregates from all UAV neighbours, again weighted by channel gain:

$$\mathbf{h}_m^{\text{UE}} = \text{MLP}\left([o_m(i) \parallel \sum_{n \in \mathcal{N}} \tilde{h}_{m,n}(i) \cdot \mathbf{h}_n^{\text{UAV}}]\right), \quad (4)$$

yielding a $d_{\text{out}} = 32$ dimensional enriched embedding per UE. The final input to the Dueling DQN is the concatenation of the raw observation and the GNN embedding:

$$\mathbf{e}_m(i) = [o_m(i) \parallel \mathbf{h}_m^{\text{UE}}] \in \mathbb{R}^{45}, \quad (5)$$

at $N = 2$ ($\text{obs_dim} = 13$, $d_{\text{out}} = 32$).

The complete architecture is shown in Fig. 2.

3.4.3 Parameter Sharing and Gradient Coupling

All M agents share a single GNN encoder, Dueling DQN, and replay buffer (capacity $M \times 5000$). Because UAV embeddings are computed via message passing from *all* UEs, a gradient update triggered by any single UE backpropagates through the shared graph structure, updating the aggregation weights for the entire network. This **gradient coupling** mechanism allows each agent to effectively train on M times more data while also learning cross-agent coordination.

Ablation experiments (Section 5.4) confirm this mechanism is critical: equipping agents with independent GNNs without parameter sharing (GNN-IL-NoShare) yields worse performance than vanilla IL. This proves that shared weights and not just embeddings enriched with spatial structure drive the performance gains.

3.5 GNN-Augmented CTDE (GNN-CTDE)

We also tested a GNN-CTDE variant, but encountered a fundamental structural conflict. The standard CTDE replay buffer stores data for one UE at a time. During training, this isolates the data, breaking the graph structure and completely eliminating the inter-UE message passing that makes the GNN effective.

Thus, GNN-CTDE empirically failed to converge within 500 episodes at $M = 20$ and $M = 30$. We exclude it from our main results, and modifying the buffer to store complete M -UE graph snapshots per timestep is left for future work.

3.6 Ablation Variants

We define four additional variants used exclusively in ablation experiments.

IL+FrameStack augments IL with temporal context by concatenating the last $K = 4$ observations into a single input vector of dimension $4 \times \text{obs_dim} = 52$. Zero-padding is applied at episode start.

GNN-IL-NoShare retains GNN-enriched observations but removes parameter sharing: each UE m maintains its own GNN encoder, its own Dueling DQN, and its own replay buffer. Gradients from UE m 's loss update only UE m 's networks.

GAT-IL replaces the weighted GraphSAGE aggregation with a graph attention mechanism [Veličković et al., 2018]. Attention is computed from UE query and UAV key vectors, then gated by the normalised channel gain before softmax, so link quality modulates learned attention.

GATPlus-IL extends GAT-IL with Prioritised Experience Replay (PER) [Schaul et al., 2016] using a SumTree for $\mathcal{O}(\log n)$ sampling, importance sampling correction weights, and an auxiliary regression loss that predicts the immediate cost $\Psi_m(i)$ from the GNN embedding:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{TD}} + \chi \cdot \text{MSE}(\hat{\Psi}_m, \Psi_m), \quad \chi = 0.5. \quad (6)$$

4 Experimental Setup

4.1 Baselines

We compare all methods against two non-learning baselines.

The **Random** policy has each UE select a processing location uniformly at random from $\mathcal{C}$ at every slot. The **Local** policy has each UE always process its task locally, never offloading.

4.2 Hyperparameters

All learning methods share the hyperparameters listed in Table 2, unless explicitly noted otherwise.

4.3 Evaluation Protocol

At the end of training, each method is evaluated over 20 deterministic episodes ($\varepsilon = 0$), and the mean cost across all UEs and all evaluation episodes is recorded as the **eval mean**. The eval standard deviation measures consistency of the learned policy. Lower eval mean indicates better performance.

Table 2: Hyperparameters shared across all learning methods

Parameter	Value	Parameter	Value
Episodes	500	Batch size	32
Steps per episode	100	Buffer capacity	5000 (per agent)
Optimizer	Adam	Target update (steps)	20
Learning rate	10^{-3}	$\varepsilon_{\text{start}}$	1.0
Discount factor γ	0.9	ε_{end}	0.05
DQN hidden dim	128	$\varepsilon_{\text{decay}}$	0.995/episode
GNN hidden dim	64	GNN output dim	32
Attention dim (GAT)	32	Aux. loss weight χ	0.5
PER α	0.6	PER β	0.4

4.4 Seeds and Experimental Runs

We distinguish two tiers of experiments.

Single-seed ablations are run at $M = 10$ with seed 42 only, and cover the baseline comparison, temporal memory ablation, $B_n(i)$ fairness analysis, parameter sharing ablation, and architecture robustness experiments. These isolate specific mechanisms under controlled conditions.

Multi-seed experiments are run with seeds $\{42, 52, 62, 72, 82\}$ across $M \in \{5, 10, 20, 30, 40\}$, comparing IL and GNN-IL to assess scalability and statistical robustness. We report mean $\pm$ standard deviation across seeds, with an exception at $M = 20$ discussed in Section 5.2.

4.5 Hardware

Single-seed ablation experiments were run on a local machine with a CUDA-enabled GPU. Multi-seed scaling experiments were run on Google Colab with a GPU runtime. Both sets of experiments use identical code and hyperparameters; however, floating-point non-determinism across different GPU hardware produces numerically distinct results for the same seed.

Concretely, at $M = 10$ seed 42, the local run yields IL = 0.6626, GNN-IL = 0.5650 (14.7% gain), while the Colab run yields IL = 0.7317, GNN-IL = 0.6252 (14.6% gain). The gain is consistent; the absolute values differ due to hardware. Single-seed ablation numbers therefore, should not be directly compared against multi-seed scaling numbers; each set is internally self-consistent.

5 Results

5.1 Baseline Comparison

Table 3 reports evaluation performance for all methods at $M = 10$, seed 42. All results in this subsection use the local hardware configuration described in Section 4.5.

Table 3: Baseline comparison at $M = 10$, seed 42. Lower eval mean is better.

Method	Eval Mean	Eval Std
Random	1.4585	—
Local	0.9513	—
CTDE (no B_n)	0.9335	0.1454
IL	0.6626	0.0681
IL ($B_n(i)$)	0.6882	0.0479
GNN-CTDE	0.6933	0.0646
CTDE ($B_n(i)$)	0.6286	0.0345
GNN-IL	0.5650	0.0279

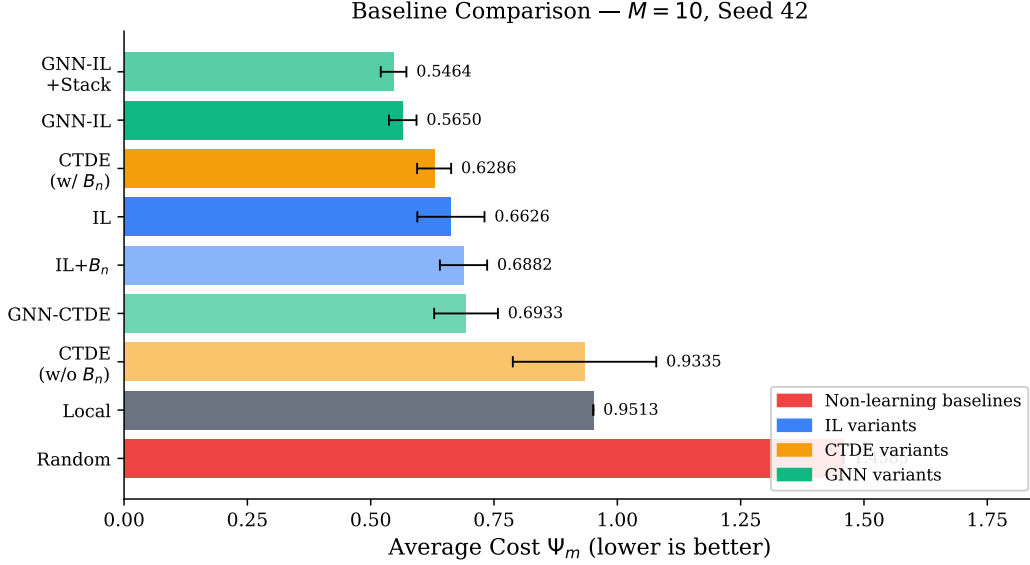


Figure 3: Baseline comparison at $M = 10$, seed 42, ordered by average cost (lower is better). GNN-IL + Stack achieves the lowest mean cost, though it remains within the margin of error of the plain GNN-IL baseline. GNN-IL achieves the lowest cost across all other methods, outperforming IL by 14.7% and CTDE (w/ B_n) by 10.1%. CTDE without $B_n(i)$ performs no better than the non-learning Local baseline, highlighting the sensitivity of shared-buffer methods to observation quality. Error bars denote evaluation standard deviation over 20 episodes.

The complete architecture is shown in Fig. 2.

GNN-IL achieves the lowest cost across all methods, reducing average cost by 14.7% over IL, 10.1% over CTDE (B_n), 40.6% over Local, and 61.3% over Random.

Notably, GNN-IL outperforms CTDE despite requiring no centralised training infrastructure; CTDE pools all agents’ experiences through a central controller during training, but GNN-IL’s graph-structured spatial awareness yields a superior policy. We attribute this to CTDE’s flat experience pooling. The shared buffer treats all M agents’ transitions as i.i.d. samples, discarding important information. GNN-IL’s bipartite graph explicitly encodes simultaneous UAV co-occupancy, giving the policy a representation of resource contention that flat pooling cannot emulate.

The result for CTDE without $B_n(i)$ is striking: it performs no better than the non-learning Local baseline (0.9335 vs. 0.9513). Adding $B_n(i)$ to CTDE’s observation recovers 32.7% in performance (0.9335 $\rightarrow$ 0.6286). This dramatic sensitivity is explained in Section 5.6 and constitutes one of the key findings of this work.

GNN-CTDE (0.6933) underperforms both CTDE (B_n) and GNN-IL. As described in Section 3.5, this is a consequence of the single-node graph limitation during training, which eliminates inter-UE message passing and the coordination signal that GNN-IL exploits. GNN-CTDE is therefore not a meaningful competitor and is excluded from further analysis.

Finally, the ordering of Random (1.4585) above Local (0.9513) is noteworthy. At $M = 10$, UAV queues are sufficiently loaded that random offloading decisions frequently incur transmission delay, making indiscriminate offloading worse than just local processing. This ordering reverses in smaller M , as explored in Section 5.2.

5.2 Scaling Analysis

Table 4 reports multi-seed performance of IL and GNN-IL across $M \in \{5, 10, 20, 30, 40\}$. These experiments were run on Google Colab; absolute values differ slightly from single-seed local results at $M = 10$ due to hardware floating-point non-determinism, but gain ratios are consistent (Section 4.5).

Fig. 4 shows mean $\pm$ std across seeds for both methods.

Fig. 5 shows rolling average training cost at $M = 10$ and $M = 30$.

At small M , UAV queues are lightly loaded and congestion is rare, meaning there is little coordination signal for the GNN to exploit. Accordingly, at $M = 5$ the two methods are nearly equivalent (2.4% mean gain), with IL even outperforming GNN-IL on 2 of 5 seeds.

As M grows, competition for UAV resources intensifies and the cost of uncoordinated offloading decisions rises sharply. When all IL agents observe a high $B_n(i)$ at UAV 1, they all independently learn that offloading to UAV 1 yields a high penalty. Because they act simultaneously without coordinating their intentions, they might all synchronously decide to switch their offloading to UAV 2 in the next time slot. This leads to oscillating, unstable policies.

GNN-IL propagates $B_n(i)$ proactively through the graph: agents see the impact of other UEs on UAV congestion before their own queues fill, enabling pre-emptive load avoidance. This advantage compounds with scale: at $M = 30$, GNN-IL achieves a 41.1% mean cost reduction over IL, the largest observed gain in this study.

Bimodal convergence. At $M = 10$ and $M = 20$, GNN-IL exhibits a bimodal convergence pattern: some seeds converge to an excellent policy while others collapse entirely. Table 5 shows per-seed results at $M = 20$, where this effect is most pronounced.

Table 5: Per-seed results at $M = 20$, illustrating bimodal convergence of GNN-IL.

Seed	IL	GNN-IL	GNN-IL status
42	0.9613	1.5061	Failed
52	0.9163	1.2366	Failed
62	0.8329	0.5988	Converged
72	0.9024	0.5998	Converged
82	0.9831	0.6193	Converged
Converged mean		0.606	
Failed mean		1.371	

When GNN-IL converges at $M = 20$, it achieves a 34.8% reduction over the IL median (0.5998 vs. 0.9163). When it fails, it performs worse than IL. There is no middle ground. We attribute this to a phase transition in the training dynamics: at $M = 20$, the system sits near a congestion threshold where the gradient signal from task-drop penalties is strong enough to train the GNN encoder reliably only if the initialisation is favourable. When initialisation is unfavourable, ε decays before the encoder has learned meaningful embeddings and training never recovers.

At larger M (30, 40), congestion is frequent and persistent (gradient signal is abundant regardless of initialisation) and GNN-IL converges on 4/5 and 4/5 seeds respectively with a tightening standard deviation (0.087 at $M = 40$).

GNN-IL therefore becomes *more* reliable as the coordination problem grows harder, not less. IL, by contrast, degrades consistently and predictably with M , its mean cost nearly doubling from $M = 10$ to $M = 30$.

We report the median as the representative statistic for $M = 20$ throughout this paper.

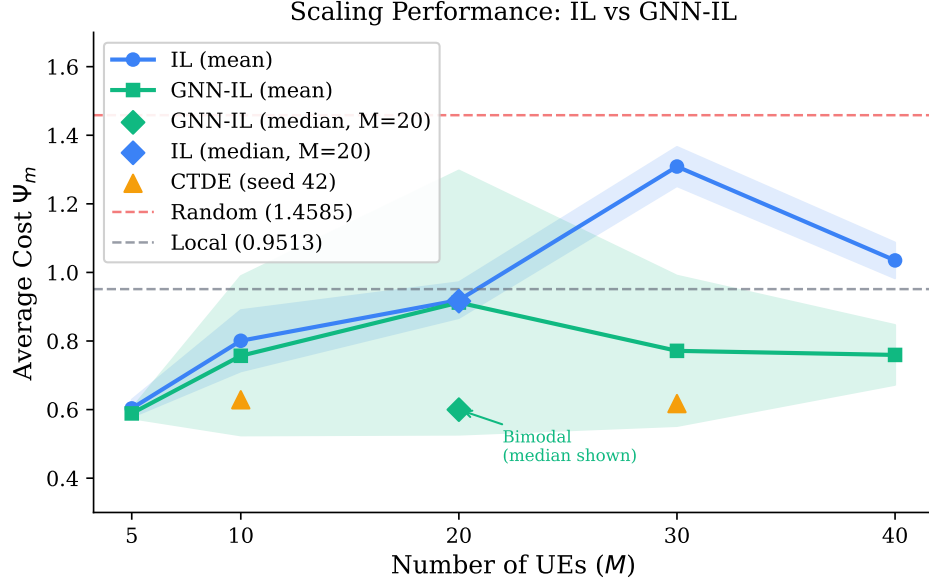


Figure 4: Scaling performance of IL and GNN-IL across $M \in \{5, 10, 20, 30, 40\}$ over 5 seeds. Shaded bands show ± 1 standard deviation. Diamond markers at $M = 20$ show per-method medians, reported in place of means due to bimodal convergence (Section 5.2). CTDE reference points (seed 42) are shown as triangles. Random and Local non-learning baselines are shown as dashed horizontal lines. GNN-IL degrades more gracefully than IL as M grows, achieving 41.1% mean cost reduction at $M = 30$. The IL cost decreases non-monotonically from $M = 30$ to $M = 40$; we attribute this to seed variance across the five runs rather than a systematic effect.

Table 4: Multi-seed scaling results (5 seeds: 42, 52, 62, 72, 82). Mean $\pm$ std reported; median shown separately for $M = 20$ due to bimodal convergence (see text). Lower is better.

M	IL mean $\pm$ std	GNN-IL mean $\pm$ std	GNN-IL median	Gain (mean)	GNN-IL wins
5	0.6035 ± 0.025	0.5888 ± 0.014	0.5872	2.4%	3/5
10	0.8007 ± 0.089	0.7569 ± 0.232	0.6252	5.5%	3/5
20	0.9192 ± 0.052	0.9121 ± 0.385	0.5998	0.8%*	3/5
30	1.3088 ± 0.057	0.7713 ± 0.219	0.6567	41.1%	5/5
40	1.0350 ± 0.052	0.7594 ± 0.087	0.7812	26.6%	5/5

* $M = 20$ mean gain is misleading due to bimodal convergence; median gain is 34.8%.

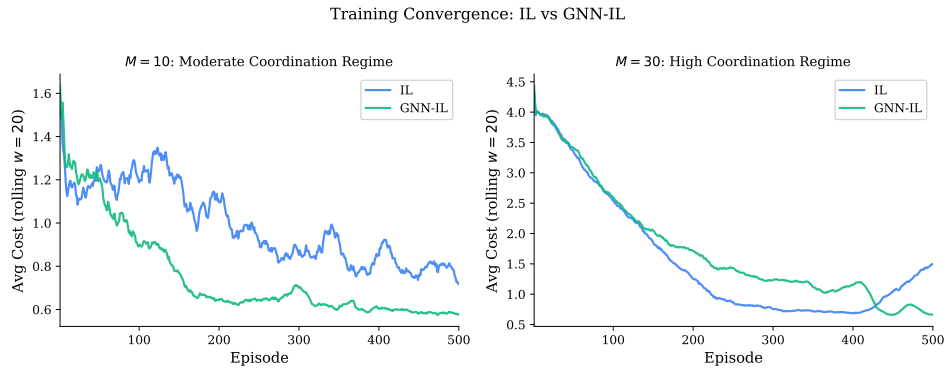


Figure 5: Training convergence curves (rolling average, window = 20) for IL and GNN-IL at $M = 10$ (left) and $M = 30$ (right), seed 42. At $M = 10$, GNN-IL converges faster and to a lower cost than IL. At $M = 30$, the gap is dramatic: GNN-IL converges to a stable low-cost policy while IL fails to find a competitive solution within the episode budget. Note: y-axis scales differ between panels.

CTDE at scale. For reference, CTDE (without B_n , seed 42) achieves 0.6286 at $M = 10$, 1.1416 at $M = 20$ (unconverged within 500 episodes; required 1000 episodes to reach 0.8038), and 0.6175 at $M = 30$. GNN-IL (seed 42) matches or beats CTDE at $M = 10$ (0.5650 vs. 0.6286) and $M = 30$ (0.6012 vs. 0.6175), while requiring no centralised training infrastructure. CTDE’s unconverged result at $M = 20$ further highlights the difficulty of the coordination problem at this scale, which GNN-IL resolves on 3/5 seeds within the same episode budget.

Random vs. Local crossover. At $M = 5$, Random (0.6305) outperforms Local (0.9176): UAVs are lightly loaded, making offloading almost always beneficial, and even random target selection yields lower cost than conservative local processing. By $M = 10$, this ordering reverses; congested UAVs make indiscriminate offloading costly.

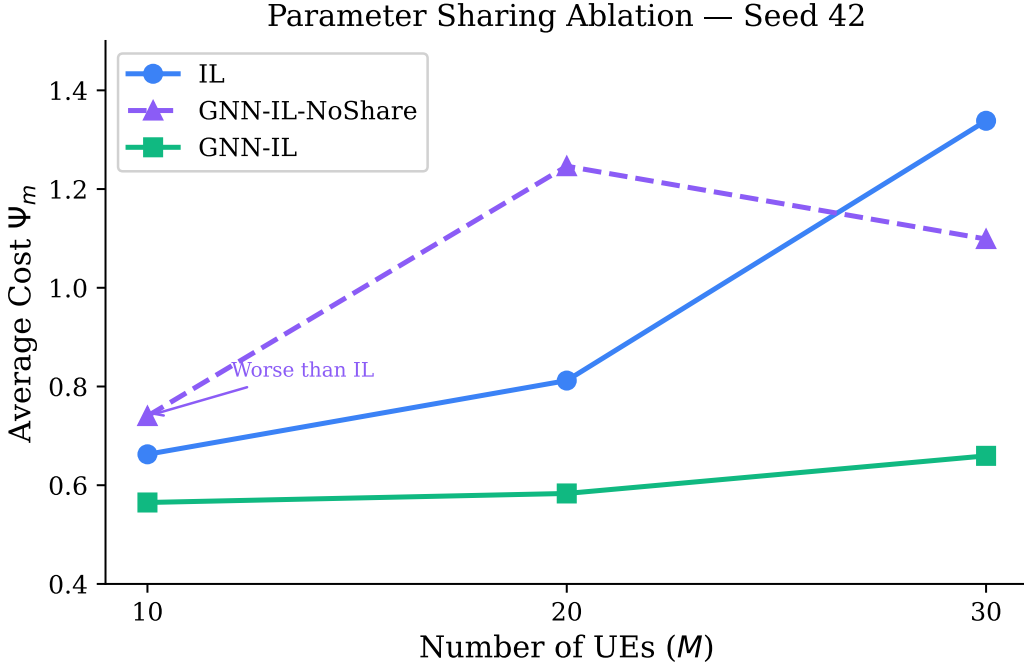


Figure 6: Parameter sharing ablation across $M \in \{10, 20, 30\}$, seed 42. GNN-IL-NoShare provides GNN-enriched observations but maintains separate networks and replay buffers per agent, removing gradient coupling. At $M = 10$ and $M = 20$, NoShare performs *worse* than vanilla IL, confirming that richer observations alone do not drive GNN-IL’s gains — shared weights and gradient coupling are the operative mechanism. At $M = 30$, NoShare partially recovers as denser task generation provides more per-agent gradient signal, but GNN-IL still dominates by 40%.

5.3 Temporal Memory Ablation

A natural hypothesis is that IL agents underperform because they lack memory of past congestion states. If agents could recall how queues evolved over recent slots, they might anticipate congestion rather than react to it.

We test this hypothesis directly using frame stacking, which concatenates the last $K = 4$ observations into a single input vector, providing the DQN with a temporal window of recent system states. Results are shown in Table 6.

Frame stacking *hurts* IL by 3.3% ($0.6626 \rightarrow 0.6847$). This could be because the queue wait features δ_m^{comp} , δ_m^{tran} , and $l_{n,m}^{\text{edge}}$ already encode temporal state implicitly by summarising the accumulated backlog in each queue. Stacking four frames quadruples the input dimensionality from 13 to 52

Table 6: Temporal memory ablation at $M = 10$, seed 42. Lower is better.

Method	Eval Mean	Eval Std
IL	0.6626	0.0681
IL + FrameStack ($K = 4$)	0.6847	0.0646
GNN-IL	0.5650	0.0279
GNN-IL + FrameStack ($K = 4$)	0.5464	0.0260

without introducing new information, making the DQN harder to train without a corresponding benefit in representational power.

Frame stacking provides a marginal improvement to GNN-IL (3.3%, $0.5650 \rightarrow 0.5464$), though the gain is small and within the noise range suggested by the standard deviation. The GNN embedding captures a spatial snapshot of congestion at the current slot; stacking slightly enriches this with information about whether congestion is building or dissipating over recent slots.

The performance gap between IL and GNN-IL is therefore not explained by temporal memory. The bottleneck is spatial coordination awareness: the ability to observe the current congestion state of shared UAV resources across all agents simultaneously, which frame stacking cannot provide and graph message passing does.

We additionally attempted a GRU-based temporal encoder for IL, implementing a sequence replay buffer with chunk length 8 and training via backpropagation through time. Training loss diverged monotonically ($1.29 \rightarrow 3.42$ over 500 episodes) without gradient clipping, and the approach was abandoned.

This is consistent with the frame stacking result: temporal memory does not address the fundamental limitation of IL’s purely local observation, regardless of the mechanism used to provide it.

5.4 Parameter Sharing Ablation

Section 5.3 established that temporal memory does not explain GNN-IL’s gains. A second candidate explanation is that GNN-IL simply provides richer observations. Agents see UAV congestion states and neighbour positions that IL agents do not.

To isolate this effect, we introduced GNN-IL-NoShare. Each UE receives GNN-enriched observations, but maintains its own separate GNN encoder, DQN, and replay buffer. Gradients from UE m ’s loss update only UE m ’s networks. If richer observations alone drive performance, GNN-IL-NoShare should match GNN-IL. Results are shown in Table 7.

Table 7: Parameter sharing ablation across M , seed 42. Lower is better.

M	IL	GNN-IL-NoShare	GNN-IL
10	0.6626	0.7401	0.5650
20	0.8120	1.2463	0.5833
30	1.3383	1.0987	0.6595

Fig. 6 visualises the three-way comparison across M .

The result is unambiguous: **GNN-IL-NoShare performs worse than vanilla IL at $M = 10$ and $M = 20$, and only approaches IL at $M = 30$.** Richer observations alone do not explain GNN-IL’s gains; they actively hurt performance when parameter sharing is removed. Training loss for GNN-IL-NoShare diverges in practice: at $M = 10$, loss grows from 2.80 to 4.63 over 500 episodes; at $M = 20$, from 5.61, barely descending to 3.26.

Each agent’s separate GNN encoder must be jointly optimised with its DQN from only 5000 transitions, not enough to stably train both the graph encoder and the downstream policy. The richer input actually increases the optimisation burden without providing the sample efficiency needed to meet it.

This points to a more precise characterisation of GNN-IL’s mechanism. Parameter sharing serves two separable roles. First, **sample efficiency**: with one shared GNN and one shared replay buffer of capacity $M \times 5000$, every agent’s transition updates the encoder that all agents use, effectively giving the encoder $M \times$ more gradient signal than a per-agent encoder receives.

Second, **implicit coordination**: when UE 1 receives a penalty and gradients flow backward through the DQN and into the GNN, they pass through the UAV aggregation node, which was computed using features from all M UEs. The gradient therefore updates the aggregation weights governing how every UE’s features influence the UAV embeddings. Thus, UE 2 is improved through UE M ’s representations as a direct consequence of UE 1’s experience. No explicit inter-agent communication is required.

At large M , each agent generates tasks more frequently (higher effective congestion), providing a denser gradient signal. The encoder becomes easier to train independently, and the richer observations begin to add value. But GNN-IL still dominates by 40% at $M = 30$, confirming that gradient coupling remains the primary driver even at scale.

The conclusion is clear: shared weights are not just a convenient implementation choice. Graph structure provides the right inductive bias for coordination, and parameter sharing provides the sample efficiency and implicit cross-agent coupling that makes that structure trainable. Neither alone is sufficient.

5.5 Architecture Robustness

Having established that parameter sharing and graph structure drive GNN-IL’s gains, we ask whether more sophisticated graph architectures can push performance further.

We evaluate two variants: GAT-IL, which replaces weighted GraphSAGE aggregation with learned attention, and GATPlus-IL, which additionally incorporates Prioritised Experience Replay and an auxiliary cost regression loss. Results are shown in Table 8.

Table 8: Architecture robustness at $M = 10$, seed 42. Lower is better.

Method	Eval Mean	Eval Std
IL	0.6626	0.0681
GNN-IL (GraphSAGE)	0.5650	0.0279
GAT-IL	0.5617	0.0255
GATPlus-IL (PER + Aux Loss)	0.5615	0.0261

All three graph-based methods perform essentially identically, with differences of less than 0.4% separate GNN-IL, GAT-IL, and GATPlus-IL. Neither learned attention nor prioritised replay nor auxiliary supervision provides a meaningful improvement over the base GraphSAGE architecture.

The GAT result is explained by the graph topology. With $N = 2$ UAV nodes, attention is computed over exactly two targets per UE. Softmax over two values with channel-gain gating is nearly equivalent to the normalised weighted aggregation that GraphSAGE already performs. The additional expressivity of learned attention has no room to manifest. Attention mechanisms earn their complexity at $N \geq 8$, where the agent must learn non-trivial selection over many candidate servers. At $N = 2$, the two approaches converge.

GATPlus addresses problems that do not exist at this scale. Task-drop events, that PER is designed to oversample, occur frequently at $M = 10$ and $N = 2$, so the standard uniform replay buffer already provides sufficient coverage of high-reward-variance transitions. The auxiliary regression loss introduces a competing optimisation objective that adds training complexity without a corresponding signal benefit.

We attempted GATPlus as a remedy for the bimodal convergence observed at $M = 20$. It converged on seed 42 (0.6045) but failed on seeds 52 (0.8938) and 82 (0.9165), confirming that it does not systematically resolve the phase transition identified in Section 5.2. Resolving bimodal convergence at $M = 20$ remains an open problem.

These results carry a positive implication: **GNN-IL’s performance gain is robust to architectural choice**. The gain arises from graph structure and parameter sharing, both of which are present in all three variants, and all three variants deliver equivalent performance. This robustness strengthens confidence that the result is not an artefact of a specific architectural configuration.

5.6 $B_n(i)$ Information Fairness Analysis

At first, $B_n(i)/M$ (the normalised active queue count at UAV n) was present in GNN-IL’s UAV node features but absent from IL’s raw observation vector. This was a problem, as GNN-IL may have been outperforming IL partly because it had access to information IL did not. We corrected this by adding $B_n(i)/M$ to IL’s observation vector (and by extension, to CTDE’s), and re-evaluated all three methods. Results are shown in Table 9.

Table 9: Effect of adding $B_n(i)/M$ to the observation vector, $M = 10$, seed 42. GNN-IL receives $B_n(i)$ implicitly via UAV node features in all configurations.

Method	Without $B_n(i)$	With $B_n(i)$	Effect
IL	0.6626	0.6882	+3.8% (worse)
CTDE	0.9335	0.6286	−32.7% (better)
GNN-IL	0.5650	—	Via graph

The results reveal a striking asymmetry: the same piece of information produces three different effects depending on the architecture.

CTDE improves dramatically. Without $B_n(i)$, CTDE performs no better than the non-learning Local baseline (0.9335 vs. 0.9513). Adding $B_n(i)$ recovers 32.7% in performance. The central controller pools transitions from all M agents simultaneously. Without $B_n(i)$, the network sees observations that look similar across agents but receives wildly different rewards depending on how many agents happened to offload to the same UAV in that slot.

From the network’s perspective, identical inputs produce contradictory outputs, making the value function nearly unlearnable. Adding $B_n(i)$ resolves this: the network can now correlate “UAV congested + offload decision $\rightarrow$ bad reward” consistently across all agents’ experiences, providing the missing context that makes the shared buffer’s pooled signal coherent.

IL is unaffected. Adding $B_n(i)$ to IL’s observation marginally *worsens* performance by 3.8%, within the range of noise. IL’s per-agent replay buffer is self-consistent from a single UE’s perspective, the agent’s own queue waits (δ_m^{comp} , δ_m^{tran} , $l_{n,m}^{\text{edge}}$) already implicitly encode the consequences of UAV congestion as experienced by that agent. Adding $B_n(i)$ introduces a global signal that a single agent cannot causally connect to its own reward variance, adding dimensionality without usable information.

GNN-IL does not need it explicitly. GNN-IL encodes $B_n(i)/M$ directly as a UAV node feature, making it available to the graph aggregation layer before the DQN ever sees the embedding. The GNN learns to propagate this congestion signal to all UE embeddings through message passing,

providing each agent with a representation of UAV load that is both richer and more structured than a flat scalar appended to the observation.

We see that **information availability is not equivalent to information utility**. The same feature ($B_n(i)$) is transformative for CTDE, irrelevant for IL, and naturally integrated for GNN-IL.

We confirm that the GNN-IL vs. IL comparison is valid: adding $B_n(i)$ to IL does not close the gap, ruling out information asymmetry as a problem.

6 Discussion

6.1 A Unified View of the Findings

The results across Sections 5.1–5.6 converge on a single coherent argument. In multi-agent computation offloading, the fundamental challenge is not temporal memory, observation richness, or architectural sophistication; it is **spatial coordination awareness**: the ability to observe the current impact of all agents on shared resources simultaneously, and to learn a policy that accounts for it.

Graph neural networks deliver this awareness through two mechanisms. Graph-structured inductive bias and parameter sharing, neither of which is sufficient alone (Section 5.4). The $B_n(i)$ analysis reinforces this: the right information in the wrong architecture is useless, while the right architecture can extract a coordination signal that a flat observation vector cannot (Section 5.6).

6.2 The Deployment Spectrum

The four methods evaluated in this work occupy distinct points on a deployment complexity spectrum, summarised in Table 10.

Table 10: Deployment characteristics of each method.

Method	Training Info	Execution Info	Eval ($M = 10$)
IL	Local only	Local only	0.6626
CTDE	Global pool	Local only	0.6286
GNN-IL	Global graph	Global graph	0.5650
GNN-CTDE	Global graph	Global graph	0.6933

IL requires no infrastructure whatsoever. Each UE trains and executes independently.

CTDE requires a central server during training but reverts to fully local execution, making it attractive for systems where a base station or cloud server is available during offline training but unavailable at deployment.

GNN-IL requires global graph construction at both training and execution time, meaning UE positions and UAV states must be broadcast to all agents at runtime.

This is a modest coordination overhead: UAVs already broadcast $B_n(i)$ in the base paper’s CTDE framework [Fatima et al., 2024], and extending this to include UE positions is lightweight relative to the task data being offloaded. The performance return - outperforming both IL and CTDE without centralised training infrastructure - justifies this cost in most practical NTN-MEC deployments.

6.3 Bimodal Convergence as a Phase Transition

The bimodal convergence of GNN-IL at $M = 10$ and $M = 20$ has practical implications for deployment. A system that either converges or fails completely, with no middle ground, is difficult to rely on without a convergence verification step. In practice, this means that deploying GNN-IL

at intermediate scales ($M \approx 20$) requires either running multiple initialisations and selecting the converged result, or adopting a strategy such as curriculum learning.

Our reasoning suggests that interventions targeting early exploration (higher initial ε , slower decay, curriculum scheduling) are more likely to resolve the instability than architectural modifications. GATPlus, which addressed buffer distribution and auxiliary supervision, did not help (Section 5.5); the problem is earlier in the training pipeline.

6.4 Limitations

We acknowledge the following limitations of this work.

No LEO satellite. The base paper’s third offloading target (a LEO satellite with $f^{\text{edge}} = 15$ GHz and propagation delay T_p) is omitted. Re-enabling LEO adds a fourth action, changes the effective action space, and would require full experimental re-runs. Its reintegration is left to future work.

N=2 UAVs. The base paper uses $N = 4$. Our choice of $N = 2$ reduces action space and simplifies graph topology, but limits the evaluation of attention mechanisms: with only two UAV nodes, GAT and GraphSAGE are nearly equivalent (Section 5.5). A proper evaluation of attention has not been done.

500 vs. 1000 episodes. We train for half the episode budget of the base paper. CTDE at $M = 20$ required 1000 episodes to converge, suggesting that some results (particularly at larger M) may improve with a longer training budget.

Single-seed CTDE. CTDE scaling results are reported for seed 42 only. Multi-seed CTDE analysis is left out due to computational cost.

GNN-IL execution overhead. Global graph construction at runtime requires knowledge of all UE positions and UAV states at each slot. While this overhead is modest in our simulation, its feasibility in real NTN-MEC deployments with mobile UEs and intermittent connectivity warrants further study.

7 Conclusion

In this work, we proposed GNN-IL: a bipartite weighted GraphSAGE encoder combined with a shared Dueling DQN, trained in a fully decentralised manner via parameter sharing across all agents. Through controlled ablation experiments, we identified the precise mechanism driving its gains and characterised its failure modes.

The key findings are as follows. **First**, GNN-IL achieves up to 41.1% cost reduction over IL at $M = 30$, and outperforms CTDE by 10.1% at $M = 10$ despite requiring no centralised training infrastructure.

Second, this gain is not explained by richer observations alone: GNN-IL-NoShare, which provides GNN-enriched observations without parameter sharing, performs *worse* than vanilla IL, confirming that shared weights (through gradient coupling and sample efficiency) are the operative mechanism.

Third, temporal memory is not the bottleneck: frame stacking hurts IL and provides only marginal benefit to GNN-IL, establishing that the limitation of independent learning is spatial, not temporal.

Fourth, the $B_n(i)$ fairness analysis reveals that information availability is not equivalent to information utility - the same congestion feature is transformative for CTDE, irrelevant for IL, and naturally integrated by GNN-IL’s graph structure.

Fifth, GNN-IL exhibits a bimodal convergence phase transition at intermediate scale ($M = 20$), becoming more reliable as M grows, while IL degrades consistently.

7.1 Future Work

Several directions follow naturally from this work.

Resolving bimodal convergence. Curriculum learning is the most promising intervention. The phase transition mechanism suggests that early exploration quality, not architectural complexity, is the limiting factor.

Transfer and zero-shot generalisation. GNN-IL’s graph-size-agnostic architecture enables a natural transfer experiment: train on $M = 10$, deploy on $M = 30$ without retraining. IL agents cannot do this, as their observation dimension is hardcoded to a fixed M . This would be a strong practical advantage of the graph-based approach.

LEO satellite re-integration. Re-enabling the LEO satellite as a third offloading target adds a fourth action ($c = N + 1$), a higher-capacity edge server ($f^{\text{edge}} = 15$ GHz), and a propagation delay T_p . The bipartite graph extends naturally to include a satellite node, and GNN-IL’s architecture requires no structural modification.

Full GNN-CTDE. A correct GNN-CTDE implementation requires storing complete M -UE graph snapshots per timestep in the replay buffer, enabling full inter-UE message passing during centralised training. This is architecturally straightforward but $M \times$ more memory-intensive, and would allow a fair comparison between graph-structured centralised and decentralised training.

Larger N and attention evaluation. Increasing to $N = 4$ (the base paper’s setting) or beyond would create a non-trivial attention problem over UAV targets, enabling a meaningful evaluation of whether learned attention over channel gains outperforms fixed weighted aggregation.

Partial offloading. Allowing each UE to split its task between local processing and edge offloading introduces a continuous action space. This represents a qualitatively richer offloading problem where GNN-IL’s coordination awareness may provide even larger gains.

References

- Nida Fatima, Pares Saxena, and Giovanni Giambene. Computation offloading in NTN-empowered MEC using multi-agent distributed deep reinforcement learning. In *GLOBECOM 2024 - 2024 IEEE Global Communications Conference*, pages 1533–1538. IEEE, 2024.
- Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in neural information processing systems*, volume 30, 2017.
- Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations*, 2016.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*, pages 1995–2003. PMLR, 2016.